

REQ 3+4 Part 2

- A. Creation of the magical orbs such as SpeedOrb, RegenerativeOrb and HeatResistantOrb that grants special effects to the actor.

Design 1:

Each orb class (SpeedOrb, RegenerativeOrb, HeatResistantOrb) inherits directly from the Item class. Each orb would independently define its own applyEffect() method and handle its own logic.

Pros	Cons
• Simple to implement, no need to design inheritance or interfaces. • Slightly easier to read for very small codebases.	• Each orb repeats common setup code (constructor logic, description text, etc.). If you want to change a shared property (e.g., pickup message), you must edit every orb class. • No enforced contract thus there's no guarantee every orb has an applyEffect() method. Developers could forget it. • Adding new shared features (like "orb glow intensity" or "durability") would require editing each orb class individually.

Design 2:

Define an abstract class MagicalOrb that contains the general structure of a magical orb. MagicalOrb inherits from Item base class then implements an OrbEffect Interface to ensure it includes the method applyEffect(Actor actor). Concrete magical orb classes such as SpeedOrb, RegenerativeOrb and HeatResistantOrb inherit from the base class MagicalOrb.

Pros	Cons
• Allows additional functionalities to be added to magical orbs beyond having special effects to actors (OrbEffect Interface), making the design more extensible for future development. • Ensures all magical orbs share the same set of methods, while allowing each magical orb to	• An interface cannot group common attributes, so each concrete class must define and initialize its own attributes. • Leads to more code duplication, as methods with identical logic must still be reimplemented in every concrete class that uses the interface.

provide its own effects, since interfaces only declare methods without defining them.	
---	--

- B. Creation of the special effects such as SpeedBoost, Regenerative and HeatResistant.

Design 1:

Implement all special effects, SpeedBoost, Regenerative, and HeatResistant, directly inside the specific actor classes (such as Player, Bear etc) as conditional flags or methods. The specific actor class itself would handle the logic for applying, updating, and removing these effects.

Pros	Cons
• Simple to implement since all logic is contained within a single class. • Easier to access or modify effects directly without creating additional classes. • Slightly faster to prototype for a small number of effects.	• Makes the codebase harder to maintain and extend, as adding new effects requires editing the actor classes directly. • Leads to code duplication if similar logic is needed for different actor types. • Reduces flexibility, effects cannot be reused or applied to other entities easily. • Increases the chance of introducing bugs when modifying or removing specific effect logic. • Makes testing more difficult since all effect behaviors are tightly coupled to the Actor implementation.

Design 2:

Create the concrete special effect classes such as SpeedBoost, Regenerative and HeatResistant and implement them with Status interface from the engine.

Pros	Cons
• Directly uses the engine's feature of automatically removing this effect after the	• Each special effect class may duplicate logic if they share similar timing or status-handling

status is not active thus no need to code additionally for this logic. • The classes are guaranteed to have methods such as <code>tickStatus()</code> and <code>isStatusActive()</code> methods.	behavior, leading to code repetition. • It may tightly couple the special effects to the engine's Status interface, reducing flexibility if the engine or effect-handling mechanism changes later.
---	---

C. Implementing the activate orb action where it activates the special effect of the magical orbs.

Design 1:

Use 'instanceof' and downcasting to check if the magical orb is in the actor's inventory and then execute the activate orb action. The logic of the activate orb action is directly implemented in the `playTurn()` method of the specific actor (e.g. Player).

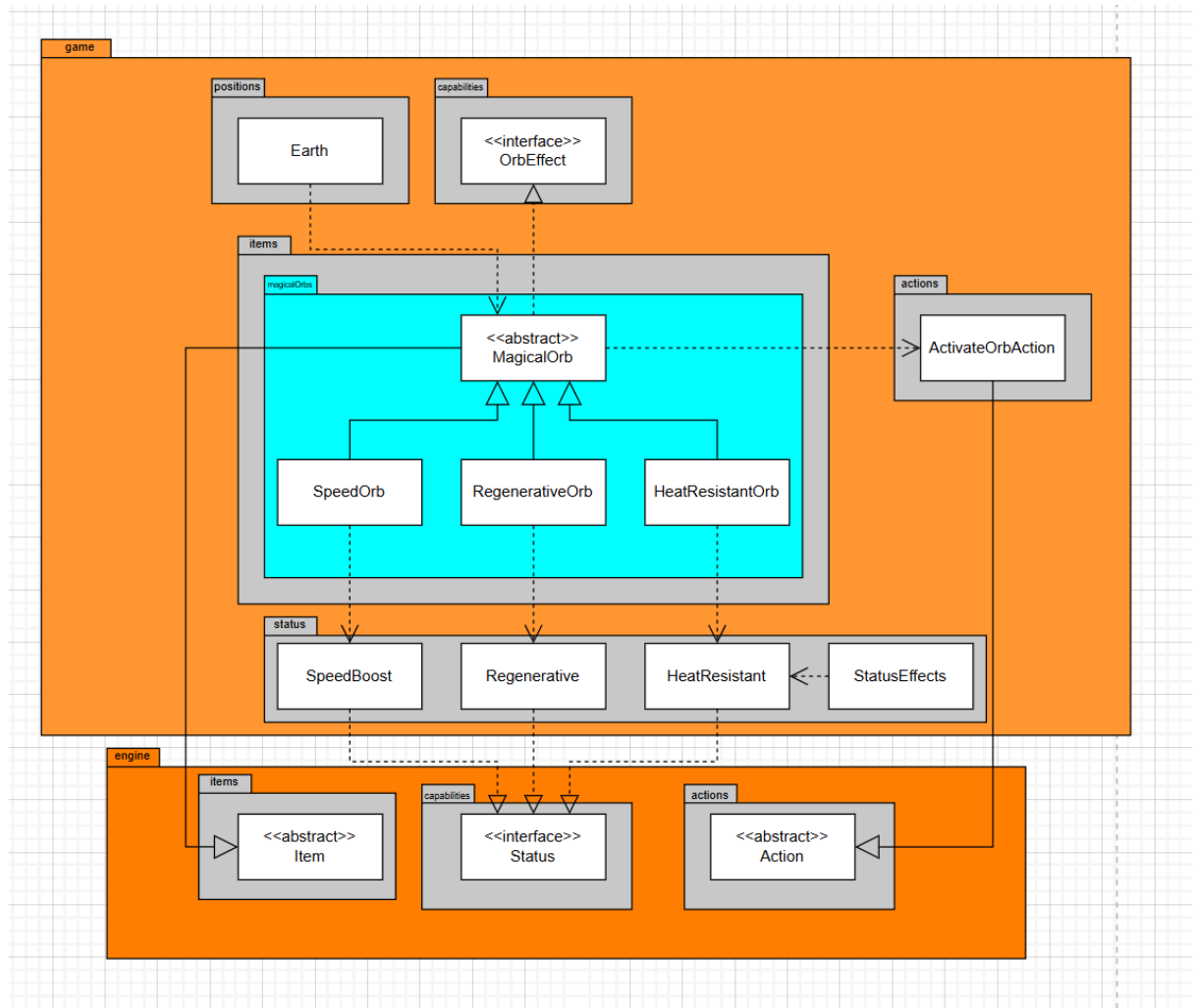
Pros	Cons
• Easy to understand and have clear implementation.	• Difficult to introduce new action types in the game, as the code must be updated with checks for each addition, which lowers clarity and maintainability. • Excessive use of 'instanceof' and downcasting is seen as a code smell because it often suggests a breach of the Open-Closed Principle. • Player class becomes a "God" class where it knows everything and has too many responsibilities

Design 2:

Create the `ActivateOrbAction` class and extend it from the given base `Action` class. Within the `MagicalOrb` class, override the `allowableActions(Actor owner, GameMap map)` method from base `Item` class so that it only provides the activate orb action when in the actor's inventory.

Pros	Cons
• Clear separation of responsibilities where <code>ActivateOrbAction</code> only manages the activate orb logic and	• Future maintainers of the code have to understand how the engine works in terms of determining if an action is

MagicalOrb only manages when activating orb is allowed. • If we add other activatable items, they can also reuse the ActivateOrbAction in their allowableActions() method.	possible based on whether the item is in the inventory or not. • MagicalOrb coupling to activate orb logic as the MagicalOrb directly returns a ActivateOrbAction .
---	--



The diagram above is my final design of REQ 3+4 (part 1). My final design uses design 2 of parts A, B and C to implement the creation of the magical orbs, the creation of the special effects and the implementation of the activate orb action.

The new abstract MagicalOrb class extends the abstract Item class, allowing it to function as a type of item that actors can interact with. The specific concrete magical orb classes such as SpeedOrb, RegenerativeOrb and HeatResistantOrb then extend from MagicalOrb. I designed it this way because abstracting these identities helps eliminate code duplication, following the DRY principle. It also implements the OrbEffect interface. Defining different traits of concrete instances through separate interfaces (e.g., consumable, item spawning, effects of orbs) assigns distinct responsibilities to each interface, improving system maintainability in line with the Single Responsibility Principle. This design also supports

future extensions, as new traits can be added by introducing additional interfaces, consistent with the Interface Segregation Principle.

The concrete special effect classes, SpeedBoost, Regenerative, and HeatResistant, implement the Status interface from the engine, ensuring they adhere to a consistent contract for managing effect behavior. I designed it this way because, by leveraging the engine's built-in status handling, the design eliminates the need to manually implement logic for tracking and removing inactive effects, reducing redundancy and aligning with the DRY principle. This approach upholds the Liskov Substitution Principle, as all effects can be treated uniformly through the Status interface without altering expected behavior. It also supports the Open/Closed Principle, allowing new effects to be introduced without modifying existing engine code. Furthermore, the consistent inclusion of methods such as tickStatus() and isActive() enhances reliability and maintainability, as all effect classes follow a predictable and well-defined lifecycle.

The ActivateOrbAction class inherits from the given base Action class. The ActivateOrbAction class controls the logic for activating the orb. I did it this way because it has a clear separation of responsibilities (Single Responsibility Principle) resulting in better maintainability in the long run instead of handling everything within the actor class. As a result, I did not have to modify any code within the Player class at all (Open-closed Principle).